SLOT 12

Form Draft Save, Focus & Keyboard Actions, HTTP Request & JSON Parsing

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

- Form Draft Save
- Focus & Keyboard Actions
- HTTP Request & JSON Parsing

1 Form Draft Save

Khái niệm quan trọng

Form Widget

Sử dụng `Form`, `TextFormField`, `GlobalKey<FormState>`

Quản lý validation và submit dữ liệu

Draft Save (Lưu bản nháp)

Lưu dữ liệu tạm thời khi người dùng chưa hoàn tất form

Có thể lưu:

In-memory (state)

Local storage (SharedPreferences)

Local database (SQLite)

Hoặc server

Form Validation

Validator function

Real-time validation

On submit validation

Mục tiêu của nội dung này

Biết cách xây dựng form có cấu trúc đúng chuẩn Flutter

Hiểu cách quản lý state của form

Biết cách lưu nháp khi:

App bị kill

User thoát màn hình

Phân biệt:

Submit chính thức

Lưu tạm thời (draft)

Hiểu tư duy UX thực tế của form trong ứng dụng lớn

2 Focus & Keyboard Actions

Khái niệm quan trọng

FocusNode

Quản lý focus giữa các input field

FocusScope

Điều hướng focus trong form

TextInputAction

next / done / search / send

Keyboard Handling

Ẩn bàn phím

Chuyển field khi nhấn "Next"

Submit khi nhấn "Done"

Mục tiêu của nội dung này

Biết điều khiển focus mượt giữa các input

PhucPT10@fpt.edu.vn

Tạo trải nghiệm nhập liệu chuyên nghiệp
Xử lý:
- Auto focus
- Next field
- Submit bằng keyboard

Tránh lỗi phổ biến:
- Memory leak do không dispose FocusNode
- Keyboard che mất UI

3 HTTP Request & JSON Parsing

Khái niệm quan trọng

HTTP Methods
- GET
- POST
- PUT
- DELETE

Asynchronous Programming
- Future
- async / await
- Error handling

JSON Parsing
- `dart:convert`
- jsonDecode()
- Model class mapping
- fromJson / toJson

Data Layer Architecture
- Repository pattern
- Separation UI & Network

Mục tiêu của nội dung này

Biết gửi request đến REST API
Biết parse JSON thành model object
Biết xử lý:
- Loading state
- Error state
- Empty state

Hiểu luồng:
`UI → Repository → API → JSON → Model → UI`
Biết tổ chức code sạch thay vì viết HTTP trực tiếp trong UI

Sau bài học Slot 12, sinh viên bắt buộc phải:

- ✅ Tự xây dựng form có validation
- ✅ Lưu nháp form khi app bị đóng
- ✅ Điều khiển focus & keyboard chuyên nghiệp
- ✅ Gọi API thật và parse JSON
- ✅ Hiểu kiến trúc tách UI & data layer

TÀI LIỆU THAM KHẢO UY TÍN

PhucPT10@fpt.edu.vn

1 📖 Tài liệu chính thức

◈ Flutter Official Documentation

Flutter

https://docs.flutter.dev

Nên đọc các mục:

- Forms and validation
- Networking
- Focus and keyboard

2 📖 Dart JSON & Async

◈ Dart Official Documentation

Dart

https://dart.dev

Nên đọc:

- Futures and async/await
- JSON and serialization

3 📘 Sách chuyên sâu

◈ "Flutter Complete Reference" – Alberto Miola

Flutter Complete Reference

Rất tốt cho:

- Form handling
- Architecture
- Networking patterns

4 🎥 Video chất lượng cao

◈ Reso Coder (YouTube Channel)

Reso Coder

Chuyên:

- Clean Architecture
- JSON parsing
- HTTP layer

5 🎥 Networking nâng cao

◈ The Net Ninja (YouTube)

The Net Ninja

Playlist:

- Flutter REST API
- Async programming

PhucPT10@fpt.edu.vn

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

1 FORM & DRAFT SAVE

◈ A. Khái niệm bắt buộc phải hiểu

Form & FormState

`Form`, `GlobalKey<FormState>`

`validate()`, `save()`, `reset()`

→ Hiểu Form là một "container quản lý state của nhiều input".

TextFormField vs TextField

→ Biết khi nào dùng `TextFormField` (có validator) thay vì `TextField`.

Validation lifecycle

Khi nào validator chạy?

Khác nhau giữa:

- onChanged
- autovalidateMode
- onSubmit

Draft Save Concept

→ Lưu dữ liệu tạm thời không phải submit chính thức.

◈ B. Kỹ thuật cần thành thạo

Tạo form có:

- validation nhiều field
- cross-field validation (vd: confirm password)

Sử dụng `TextEditingController`

Lưu nháp:

- In memory
- Local storage (vd SharedPreferences)

Load lại draft khi mở màn hình

Xử lý khi user thoát màn hình (WillPopScope)

◈ C. Lỗi thường gặp

❌ Không dispose controller → memory leak

❌ Viết logic lưu draft trong UI quá nhiều

❌ Validate nhưng vẫn cho submit

❌ Không phân biệt:

- Draft save
- Final submit

◈ D. Best Practices

Tách:

- UI layer
- Draft logic

Debounce khi auto-save

Không auto-save mỗi ký tự (tốn tài nguyên)

UX rõ ràng: hiển thị "Draft saved"

2 FOCUS & KEYBOARD ACTIONS

◈ A. Khái niệm bắt buộc phải hiểu

FocusNode

→ Quản lý trạng thái focus của input.

PhucPT10@fpt.edu.vn

FocusScope
→ Điều hướng giữa các field.
TextInputAction
next
done
search
send
Keyboard lifecycle
→ Bàn phím mở, đóng, submit.

B. Kỹ thuật cần thành thạo

Tạo nhiều FocusNode
Chuyển focus khi nhấn Next
Submit khi nhấn Done
Ẩn keyboard khi tap ngoài
Auto scroll khi keyboard che input
Quản lý layout với `SingleChildScrollView`

C. Lỗi thường gặp

❌ Không dispose FocusNode
❌ Keyboard che mất nút submit
❌ Nhấn "Next" nhưng không chuyển field
❌ Không xử lý submit từ keyboard

D. Best Practices

Dùng `FocusScope.of(context).nextFocus()`
Dùng `resizeToAvoidBottomInset`
Dùng `MediaQuery.of(context).viewInsets.bottom`
Trải nghiệm nhập liệu phải "mượt như app thật"

3 HTTP REQUEST

A. Khái niệm bắt buộc phải hiểu

HTTP methods
GET
POST
PUT
DELETE
Status codes
200 (success)
400 (bad request)
401 (unauthorized)
500 (server error)
Asynchronous programming
Future
async/await
try/catch

B. Kỹ thuật cần thành thạo

Gửi GET request
Gửi POST với body JSON

PhucPT10@fpt.edu.vn

Set header (Content-Type, Authorization)
Xử lý:
- loading state
- error state
- empty state

Timeout handling

C. Lỗi thường gặp

❌ Gọi API trực tiếp trong build()
❌ Không xử lý lỗi
❌ Không await → UI lỗi
❌ Không xử lý null JSON

D. Best Practices

Tách:

```
UI → Repository → API Service
```

Không viết HTTP trong Widget
Centralized error handling
Logging request khi debug

4 JSON PARSING & MODEL MAPPING

A. Khái niệm bắt buộc phải hiểu

JSON structure:
- Map<String, dynamic>
- List<dynamic>

Model class:
- fromJson()
- toJson()

Serialization vs Deserialization

B. Kỹ thuật cần thành thạo

Dùng `jsonDecode`
Map JSON → Dart object
Parse list object
Xử lý null-safe model
Optional fields

C. Lỗi thường gặp

❌ Ép kiểu sai
❌ Không check null
❌ Hardcode key sai tên
❌ Parse list nhưng quên map từng phần tử

D. Best Practices

Tạo model rõ ràng
Không truyền Map raw vào UI
Dùng factory constructor
Tách model vào thư mục riêng

PhucPT10@fpt.edu.vn

5 TƯ DUY KIẾN TRÚC (CỰC KỲ QUAN TRỌNG)

A. Khái niệm cần hiểu

Separation of Concerns
Clean layering
Single Responsibility Principle

B. Kỹ năng bắt buộc

Không để:
- UI xử lý HTTP
- UI parse JSON

Viết Repository pattern đơn giản
Quản lý trạng thái:
- loading
- success
- error

C. Lỗi thường gặp

❌ Code tất cả trong 1 file
❌ Logic trộn UI
❌ Không xử lý trạng thái mạng

D. Best Practices

Folder structure rõ ràng:

```
models/
services/
repositories/
screens/
```

Viết code có thể mở rộng
Tư duy: "Nếu project 100 màn hình thì sao?"

SAU SLOT 12 – SINH VIÊN PHẢI LÀM ĐƯỢC

✓ Tự xây form nhiều field có validation
✓ Lưu draft và load lại dữ liệu
✓ Điều khiển focus & keyboard chuyên nghiệp
✓ Gọi API thật và xử lý lỗi
✓ Parse JSON thành model object
✓ Tổ chức code theo layer cơ bản

PhucPT10@fpt.edu.vn

III. ỨNG DỤNG THỰC TIỄN

🎯 TÌNH HUỐNG 1 – Ứng dụng E-Commerce: Form Checkout có Draft Save

📌 Bối cảnh thực tế

Một app thương mại điện tử tương tự
Shopee
hoặc
Tiki
Người dùng nhập:
- Họ tên
- SĐT
- Địa chỉ
- Ghi chú

Nhưng:
- Họ thoát app giữa chừng
- App bị kill
- Mất mạng

❗ Vấn đề gặp phải

- User quay lại phải nhập lại từ đầu
- Tỉ lệ bỏ giỏ hàng cao
- UX kém chuyên nghiệp

🛠 Kỹ thuật được áp dụng

1 Form + Validation
- Dùng `Form`, `TextFormField`
- Validate số điện thoại, địa chỉ

2 Draft Save
- Khi user nhập → auto-save sau 1–2 giây (debounce)
- Lưu vào local storage
- Khi mở lại màn hình → load draft

3 Focus & Keyboard
- Nhấn "Next" → chuyển field
- Nhấn "Done" → submit
- Scroll tránh keyboard che nút “Đặt hàng”

🎯 Mục tiêu khi áp dụng

- Giảm friction khi thanh toán
- Tăng conversion rate
- Cải thiện UX chuyên nghiệp

🏆 Kết quả

- Giảm 18% tỉ lệ bỏ giỏ hàng
- Tăng completion rate form

📚 Bài học rút ra

✓ Draft Save không phải tính năng phụ — nó là tính năng giữ doanh thu
✓ UX nhập liệu quyết định conversion
✓ Form phải được thiết kế như một quy trình sản phẩm thật

PhucPT10@fpt.edu.vn

🎯 TÌNH HUỐNG 2 – Ứng dụng Tuyển dụng: CV Online Editor

📌 Bối cảnh thực tế

Một nền tảng tuyển dụng như
LinkedIn
Cho phép người dùng chỉnh sửa:
- Kinh nghiệm
- Học vấn
- Kỹ năng

❗ Vấn đề

- CV dài nhiều phần
- User mất 20–30 phút để chỉnh sửa
- Nếu thoát giữa chừng → mất toàn bộ dữ liệu

🛠 Kỹ thuật được áp dụng

1 Multi-section Form

Mỗi section là một Form riêng

2 Draft Save từng phần

Save mỗi section khi user rời khỏi phần đó
Lưu local + sync server khi có mạng

3 HTTP + JSON

GET profile
Parse JSON → model
PUT update từng phần

🎯 Mục tiêu

Không bao giờ để user mất dữ liệu
Tăng độ tin cậy nền tảng

🏆 Kết quả

Tăng retention user
Giảm complaint mất dữ liệu

📚 Bài học

✓ Không được phụ thuộc hoàn toàn vào server
✓ Draft local là lớp bảo vệ UX
✓ JSON model phải rõ ràng để update từng phần

🎯 TÌNH HUỐNG 3 – Ứng dụng Booking: Tìm kiếm & API Request

📌 Bối cảnh thực tế

Ứng dụng đặt phòng như
Booking.com
User nhập:
- Điểm đến
- Ngày check-in
- Ngày check-out

❗ Vấn đề

Khi user nhấn search → API call

PhucPT10@fpt.edu.vn

Nếu không xử lý async đúng → UI lag
Nếu không xử lý error → app crash

🛠 Kỹ thuật được áp dụng

1 HTTP GET Request
Gửi query params
Xử lý timeout

2 Loading State
Hiển thị spinner
Disable nút search khi đang load

3 JSON Parsing
Parse list khách sạn
Map thành model object

4 Error Handling
401 → redirect login
500 → show retry button

🎯 Mục tiêu
UI không bị freeze
Người dùng hiểu app đang xử lý
Không crash khi mạng yếu

🏆 Kết quả
UX mượt mà
Tăng độ tin cậy app

📚 Bài học
✓ Không bao giờ gọi API trong build()
✓ Luôn có 3 state: loading – success – error
✓ Phải thiết kế UI cho tình huống mạng yếu

🎯 TÌNH HUỐNG 4 – Ứng dụng Giáo dục: Bài thi online

📌 Bối cảnh
Ứng dụng học trực tuyến như
Coursera
Sinh viên làm bài test 30 phút.

❗ Vấn đề
Mạng chập chờn
User thoát app
App crash

🛠 Áp dụng kỹ thuật

1 Draft Save mỗi câu trả lời
Auto-save mỗi khi chọn đáp án

2 Sync định kỳ lên server
POST answers
Retry nếu fail

PhucPT10@fpt.edu.vn

3 JSON Mapping

Parse đề thi từ API
Map câu hỏi → model

Mục tiêu

Không mất bài thi
Không gian lận
Tăng độ tin cậy

Bài học

✓ Auto-save là tính năng bắt buộc
✓ Offline-first thinking rất quan trọng
✓ API phải thiết kế idempotent

TÌNH HUỐNG 5 – Ứng dụng Chat: Focus & Keyboard UX

Bối cảnh

Ứng dụng nhắn tin như
Zalo

Vấn đề

Keyboard mở che message
Không tự focus input
Không gửi khi nhấn Enter

Áp dụng kỹ thuật

AutoFocus khi mở room chat
TextInputAction.send
Submit khi nhấn Enter
Scroll lên khi keyboard mở

Mục tiêu

UX mượt như app thật
Tối ưu thao tác người dùng

Bài học

✓ Keyboard UX quyết định chất lượng app
✓ FocusNode phải quản lý đúng lifecycle

KẾT NỐI KIẾN THỨC → THỰC TẾ

Kỹ thuật	Ứng dụng thực tế	Giá trị mang lại
Form Validation	Checkout, đăng ký	Giảm lỗi nhập liệu
Draft Save	CV editor, bài thi	Không mất dữ liệu
Focus Control	Chat, form dài	UX chuyên nghiệp
HTTP Request	Search, booking	Kết nối backend
JSON Parsing	Tất cả API	Dữ liệu cấu trúc rõ ràng

KẾT LUẬN QUAN TRỌNG

Slot 12 không chỉ là:
"Biết gọi API và làm form"

PhucPT10@fpt.edu.vn

Mà là:
“Biết xây dựng một feature có UX thật, dữ liệu thật và hành vi production”

PhucPT10@fpt.edu.vn

IV. MỘT SỐ NỘI DUNG CHI TIẾT

◈ 1 FORM & DRAFT SAVE – PHÂN TÍCH CHUYÊN SÂU

I. NỀN TẢNG & NGUYÊN LÝ

1 Form trong Flutter thực chất là gì?

`Form` là một widget dùng để:

Gom nhóm nhiều input
Quản lý trạng thái validate
Cho phép thao tác tập trung qua `FormState`

Bên trong:

`Form` lưu danh sách các `FormField`
Khi gọi `validate()` → duyệt toàn bộ child fields

Nguyên lý hoạt động:

```
Form
 ├── TextFormField (FormField<String>)
 ├── TextFormField
 └── ...
```

`TextFormField` là một wrapper của:

`TextField`
`FormField`

II. CHỦ ĐIỂM CHUYÊN SÂU

◈ 1. Validation Strategy

Có 3 kiểu validation:

Kiểu	Đặc điểm	Khi dùng
On Submit	validate() khi bấm nút	Form dài
Realtime	onChanged	UX hiện đại
AutoValidate	autovalidateMode	Login

👉 Production thường kết hợp cả 3.

◈ 2. Cross-field Validation

Ví dụ:

Password & Confirm Password

Phải:

Lưu giá trị field A
So sánh với field B

⚠ Sai lầm thường gặp:

Validate từng field độc lập → không đủ logic

◈ 3. Draft Save – Nguyên lý thật sự

Draft Save không chỉ là "lưu local".
Nó là tư duy:
"User data không được phép mất"
Có 3 mức độ:

Mức	Lưu ở đâu	Khi dùng
Level 1	In memory	Screen nhỏ
Level 2	Local storage	Form dài
Level 3	Server sync	App lớn

PhucPT10@fpt.edu.vn

III. VẤN ĐỀ THỰC TIỄN

1 Auto-save quá thường xuyên

Nếu save mỗi ký tự:
- Tốn I/O
- Giảm performance

Giải pháp:
- Debounce 1–2 giây
- Save khi mất focus

2 Conflict khi sync server

Tình huống:
- User mở 2 thiết bị
- Cả hai đều chỉnh sửa draft

Giải pháp:
- Timestamp
- Versioning

IV. BEST PRACTICES CHUYÊN NGHIỆP

- Tách DraftService
- Không viết save logic trong UI
- Lưu theo JSON structure
- Có cơ chế clear draft khi submit thành công

V. MỞ RỘNG CHUYÊN SÂU

- Offline-first architecture
- Conflict resolution
- Event-driven save

◈ 2 FOCUS & KEYBOARD – CƠ CHẾ BÊN TRONG

I. NỀN TẢNG

Flutter sử dụng:
- FocusNode
- FocusScopeNode
- TextInputClient

Khi user tap vào field:
- FocusNode nhận focus
- Flutter gửi signal đến platform
- Platform mở keyboard

II. CHỦ ĐIỂM QUAN TRỌNG

◈ 1. Focus Lifecycle

```
initState → create FocusNode
dispose → dispose FocusNode
```

Không dispose:
→ memory leak

◈ 2. Focus Traversal

```
FocusScope.of(context).nextFocus()
```

Production thường:
- Tùy biến traversal order
- Ẩn keyboard khi chạm ngoài

PhucPT10@fpt.edu.vn

III. VẤN ĐỀ THỰC TẾ

1 Keyboard che nút submit

Giải pháp:

- SingleChildScrollView
- resizeToAvoidBottomInset
- MediaQuery.viewInsets

2 UX không mượt

Ví dụ app như
Zalo
Nếu:

- Không autoFocus
- Không scroll đúng

→ Cảm giác app "rẻ tiền"

IV. BEST PRACTICES

- Dùng `TextInputAction.next`
- Submit bằng keyboard
- Tự viết keyboard dismiss widget

V. MỞ RỘNG

- Custom keyboard
- Hardware keyboard support
- Accessibility focus management

◈ 3 HTTP REQUEST – NỀN TẢNG MẠNG

I. NGUYÊN LÝ HTTP

HTTP là:

- Stateless
- Request → Response

Production app như
Booking.com
phải xử lý:

- Timeout
- Retry
- Token expired

II. ASYNC TRONG FLUTTER

Event loop:

```
Main Thread
  → Future
  → Await
  → Resume
```

Sai lầm:

- Không await
- Gọi API trong build()

III. CÁC CHỦ ĐIỂM NÂNG CAO

◈ 1. Error Handling Strategy

try/catch

PhucPT10@fpt.edu.vn

Custom Exception
Central error handler

◈ 2. Timeout & Retry

Production cần:

Timeout 10–30s
Retry 2–3 lần

◈ 3. Authentication Flow

Access token
Refresh token
Interceptor

IV. PRODUCTION PROBLEMS

Server trả JSON thiếu field
API thay đổi version
Network yếu

Giải pháp:

Null-safe model
API versioning
Graceful degradation

◈ 4 JSON PARSING – CHUYÊN SÂU DATA MODELING

I. NGUYÊN LÝ

JSON → Map<String, dynamic>
Map → Dart Object

II. TẠI SAO PHẢI TẠO MODEL?

Nếu không:

```
response["user"]["name"]
```

→ Khó bảo trì
→ Dễ lỗi

Model giúp:

Type safety
Autocomplete
Clean code

III. CHỦ ĐIỂM QUAN TRỌNG

◈ 1. Nested JSON

```
{
  "user": {
    "id": 1,
    "profile": { ... }
  }
}
```

Phải parse nhiều lớp model.

◈ 2. List Parsing

```
(json['items'] as List)
    .map((e) => Item.fromJson(e))
    .toList();
```


PhucPT10@fpt.edu.vn

Sai lầm:

- Quên cast
- Không check null

IV. PRODUCTION ISSUES

- Backend đổi key
- Field optional
- Backend trả null

Giải pháp:

- Default value
- Nullable field
- Defensive parsing

V. MỞ RỘNG

- Code generation
- Immutable model
- Freezed pattern

5 TƯ DUY KIẾN TRÚC – MỨC ĐỘ CHUYÊN NGHIỆP

Production app như
LinkedIn
không bao giờ:

- Gọi HTTP trong UI
- Parse JSON trong Widget

Phải có:

```
UI
 ↓
ViewModel / Controller
 ↓
Repository
 ↓
API Service
```

KẾT LUẬN CHUYÊN SÂU

Slot 12 thực chất dạy 3 tư duy lớn:

1. UX không được làm mất dữ liệu
2. Network không bao giờ ổn định
3. Data phải có cấu trúc rõ ràng

Nếu sinh viên hiểu sâu các nguyên lý này, họ có thể:

- Xây dựng app production thật
- Làm việc backend team
- Mở rộng sang Clean Architecture

PhucPT10@fpt.edu.vn

V. KINH NGHIỆM THỰC TIỄN, TIPS

1 FORM & DRAFT SAVE

KINH NGHIỆM THỰC TẾ

1 Draft Save không phải "nice-to-have"

Trong các app như
Shopee
hoặc
Tiki
Form checkout nếu mất dữ liệu:
    Tỉ lệ rời bỏ tăng mạnh
    Người dùng bực bội
👉 Kinh nghiệm thực tế:
    Form dài hơn 5 field → nên có draft
    Form mất > 1 phút nhập → bắt buộc có draft

2 Không phải lúc nào cũng auto-save

Sai lầm junior:
    Lưu mỗi ký tự → app giật
Kinh nghiệm:
    Debounce 800ms – 2s
    Hoặc save khi:
        onEditingComplete
        mất focus

3 Phải clear draft đúng lúc

Rất nhiều app quên:
    Submit thành công
    Nhưng draft local vẫn còn
Hậu quả:
    Mở lại form → load dữ liệu cũ
    User tưởng hệ thống lỗi

✕ LỖI PHỔ BIẾN

    Không dispose TextEditingController
    Viết toàn bộ validation trong UI
    Không phân biệt:
        Validation lỗi client
        Validation lỗi server
    Không xử lý trường hợp:
        Server reject dữ liệu

☑ BEST PRACTICES

✓ Tách DraftService
✓ Tách Validation logic
✓ Log lại form submission khi debug
✓ UX hiển thị trạng thái: "Draft saved"
✓ Không auto-save khi form chưa hợp lệ

2 FOCUS & KEYBOARD

PhucPT10@fpt.edu.vn

🧠 KINH NGHIỆM THỰC TẾ

1 UX nhập liệu quyết định chất lượng app

Trong app chat như
Zalo
Nếu:
- Keyboard che nội dung
- Không gửi bằng Enter
- Không auto-focus

→ Cảm giác app "thiếu chuyên nghiệp"

2 FocusNode dễ gây memory leak

Junior thường:
- Tạo FocusNode
- Không dispose

Khi màn hình push/pop nhiều lần:
→ RAM tăng dần

3 Keyboard handling phải test trên nhiều thiết bị

Thực tế:
- Android & iOS xử lý keyboard khác nhau
- Thiết bị màn hình nhỏ dễ lỗi layout

❌ LỖI PHỔ BIẾN

- Không xử lý tap ngoài để ẩn keyboard
- Gọi requestFocus trong build()
- Không set TextInputAction
- Không scroll khi keyboard mở

✅ BEST PRACTICES

✓ Dispose tất cả FocusNode
✓ Sử dụng FocusScope thay vì requestFocus thủ công nhiều nơi
✓ Kiểm thử trên:
- Android nhỏ
- iPhone màn hình bé
  - ✓ Tạo reusable KeyboardDismissWidget

🔷 3 HTTP REQUEST

🧠 KINH NGHIỆM THỰC TẾ

1 Network không bao giờ ổn định

App như
Booking.com
Phải xử lý:
- Timeout
- Server chậm
- JSON thiếu field

👉 Kinh nghiệm:
Không bao giờ giả định:
`API sẽ luôn trả đúng`

PhucPT10@fpt.edu.vn

2 Không bao giờ gọi API trong build()

Sai lầm rất phổ biến:

```
@override
Widget build(BuildContext context) {
  fetchData(); // ❌
}
```

→ Mỗi rebuild gọi API lại
→ Spam request

3 Luôn có 3 trạng thái

Loading
Success
Error

Nếu thiếu 1 trong 3:
→ UX kém hoặc crash

❌ LỖI PHỔ BIẾN

Không await Future
Không catch exception
Không kiểm tra status code
Hardcode URL trong nhiều file
Không xử lý 401 (token hết hạn)

✅ BEST PRACTICES

✓ Tách API Service
✓ Tách Repository
✓ Centralized error handling
✓ Dùng timeout()
✓ Log request/response khi debug
✓ Không expose raw JSON cho UI

4 JSON PARSING

KINH NGHIỆM THỰC TẾ

1 Backend luôn thay đổi

Trong dự án thật:

Backend đổi tên field
Thêm field mới
Xóa field cũ

Nếu model không linh hoạt:
→ App crash

2 Null-safe là bắt buộc

Junior thường:

Ép kiểu thẳng
Không check null

Production:

Tất cả field phải nullable nếu backend có thể trả null

3 Không parse JSON trong UI

Sai lầm:

PhucPT10@fpt.edu.vn

```
Text(response["user"]["name"])
```

Hậu quả:

- Khó bảo trì
- Khó refactor
- Không type-safe

❌ LỖI PHỔ BIẾN

- Quên cast List
- Không xử lý nested JSON
- Không dùng default value
- Không bắt lỗi format date

✅ BEST PRACTICES

✓ Tạo model riêng
✓ Dùng factory constructor
✓ Defensive parsing
✓ Có test cho model parsing
✓ Tách thư mục models/

5 KIẾN TRÚC & TƯ DUY PRODUCTION

KINH NGHIỆM THỰC TẾ

App lớn như
LinkedIn
Không bao giờ:

- Để UI gọi thẳng HTTP
- Parse JSON trong Widget
- Không xử lý offline

❌ LỖI PHỔ BIẾN CỦA JUNIOR

- Tất cả code trong 1 file
- Không phân tầng
- Không logging
- Không retry khi fail
- Không cache

✅ BEST PRACTICES CHUYÊN NGHIỆP

✓ Layered architecture:

```
UI
 ↓
Controller/ViewModel
 ↓
Repository
 ↓
API Service
```

✓ Tách rõ:

- Business logic
- Presentation logic
- Data logic

✓ Luôn nghĩ:
Nếu 100 màn hình thì sao?

PhucPT10@fpt.edu.vn

MẸO THỰC CHIẾN (GOLD TIPS)

1 Luôn test trong điều kiện xấu

Mạng chậm
Mất mạng
App kill giữa chừng

2 Viết code như backend sẽ sai

Không tin tưởng API hoàn toàn.

3 Luôn xử lý lifecycle

dispose controller
cancel request nếu cần
tránh memory leak

4 Thiết kế UI cho trạng thái lỗi

Error UI phải:

Có retry
Có thông báo rõ ràng
Không crash

TỔNG KẾT TƯ DUY TRƯỞNG THÀNH

Sau Slot 12, sinh viên nên đạt được tư duy:

1 Data có thể mất → phải bảo vệ
2 Network có thể fail → phải xử lý
3 User có thể làm sai → phải validate
4 Backend có thể đổi → phải defensive code

.

PhucPT10@fpt.edu.vn

EXERCISE

BÀI TẬP TỔNG HỢP

Profile Editor App (Có Draft + API)

1 TÓM TẮT DỰ ÁN

Tên dự án

Smart Profile Editor

Mục tiêu

Xây dựng một màn hình chỉnh sửa hồ sơ người dùng:

Load dữ liệu từ API
Hiển thị vào form
Cho phép chỉnh sửa
Lưu nháp tự động
Submit cập nhật lên server
Xử lý loading / error / keyboard UX

2 CHỨC NĂNG CHÍNH

- ✅ Load profile từ API (GET)
- ✅ Parse JSON thành Model
- ✅ Hiển thị dữ liệu trong Form
- ✅ Validation (email, phone)
- ✅ Focus chuyển field mượt
- ✅ Draft Save local
- ✅ Submit (PUT request)
- ✅ Loading state
- ✅ Error state
- ✅ Clear draft khi submit thành công

3 CÔNG NGHỆ SỬ DỤNG

Flutter (Stable)
HTTP package
dart:convert
SharedPreferences
async/await
null-safety

4 CẤU TRÚC THƯ MỤC

```
lib/
 ├── models/
 │     profile_model.dart
 │
 ├── services/
 │     api_service.dart
 │     draft_service.dart
 │
 ├── repositories/
 │     profile_repository.dart
 │
 ├── screens/
 │     profile_screen.dart
```


PhucPT10@fpt.edu.vn

```
└── main.dart
```

5 TÓM TẮT CÁC BƯỚC THỰC HIỆN

Tạo project
Cài dependencies
Tạo Model
Tạo API Service
Tạo Repository
Tạo Draft Service
Xây UI Form
Thêm Focus & Keyboard handling
Thêm Draft auto-save
Thêm Submit logic
Thêm loading & error state
Test toàn bộ flow

◈ HƯỚNG DẪN CHI TIẾT

0) Chuẩn bị trước khi làm (bắt buộc)

0.1. Kiểm tra môi trường

Mở Terminal/Command Prompt, chạy:

```bash
flutter --version
flutter doctor
```

Kiểm chứng

`flutter doctor` không còn lỗi đỏ (ít nhất Android toolchain/Chrome/iOS tùy máy phải OK).
Nếu có lỗi: sửa theo gợi ý trong `flutter doctor`.

1) BƯỚC 1 — Tạo project + chạy lần đầu

1.1. Tạo project

```bash
flutter create smart_profile_editor
cd smart_profile_editor
```

1.2. Chạy app

```bash
flutter run
```

Kiểm chứng

App mở lên thấy màn hình counter mặc định.
Không crash.

Lỗi hay gặp

"No devices": bạn chưa mở emulator / chưa cắm điện thoại / chưa bật USB debugging.

2) BƯỚC 2 — Cài dependencies

Mở `pubspec.yaml` và thêm:

```YAML
dependencies:
  flutter:
    sdk: flutter
```


PhucPT10@fpt.edu.vn

```
  http: ^0.13.6
  shared_preferences: ^2.2.2
```

Sau đó chạy:

```bash
flutter pub get
```

Kiểm chứng

- Không báo lỗi version conflict.
- Run lại vẫn chạy.

3) BƯỚC 3 — Tạo cấu trúc thư mục chuẩn

Trong thư mục `lib/`, tạo các folder:

```
models/
services/
repositories/
screens/
```

Kiểm chứng

- Cây thư mục giống yêu cầu đề bài.

4) BƯỚC 4 — Tạo Model (Profile)

Tạo file dưới đây.

lib/models/profile_model.dart

```dart
class Profile {
  final String name;
  final String email;
  final String phone;

  const Profile({
    required this.name,
    required this.email,
    required this.phone,
  });

  Profile copyWith({
    String? name,
    String? email,
    String? phone,
  }) {
    return Profile(
      name: name ?? this.name,
      email: email ?? this.email,
      phone: phone ?? this.phone,
    );
  }

  factory Profile.fromJson(Map<String, dynamic> json) {
    return Profile(
      name: (json['name'] ?? '').toString(),
      email: (json['email'] ?? '').toString(),
      phone: (json['phone'] ?? '').toString(),
    );
  }
```


PhucPT10@fpt.edu.vn

```
  Map<String, dynamic> toJson() {
    return {'name': name, 'email': email, 'phone': phone};
  }
}
```

Kiểm chứng

- `flutter analyze` không lỗi.
- App vẫn build được.

5) BƯỚC 5 — API Service (GET + PUT) + có log để kiểm chứng

Tạo file:

lib/services/api_service.dart

```
import 'dart:convert';
import 'package:http/http.dart' as http;

import '../models/profile_model.dart';

class ApiService {
  // JSONPlaceholder: giả lập user id=1
  final String baseUrl = "https://jsonplaceholder.typicode.com/users/1";

  Future<Profile> fetchProfile() async {
    final uri = Uri.parse(baseUrl);
    final response = await http.get(uri);

    // Log để SV thấy rõ
    // ignore: avoid_print
    print("GET $uri -> ${response.statusCode}");
    // ignore: avoid_print
    print("Body: ${response.body}");

    if (response.statusCode == 200) {
      final jsonMap = jsonDecode(response.body) as Map<String, dynamic>;
      return Profile.fromJson(jsonMap);
    } else {
      throw Exception("Failed to load profile (${response.statusCode})");
    }
  }

  Future<void> updateProfile(Profile profile) async {
    final uri = Uri.parse(baseUrl);

    final response = await http.put(
      uri,
      body: jsonEncode(profile.toJson()),
      headers: {'Content-Type': 'application/json'},
    );

    // ignore: avoid_print
    print("PUT $uri -> ${response.statusCode}");
    // ignore: avoid_print
    print("Body: ${response.body}");
```


PhucPT10@fpt.edu.vn

```
    // JSONPlaceholder thường trả 200.
    if (response.statusCode != 200) {
      throw Exception("Failed to update profile (${response.statusCode})");
    }
  }
}
```

Kiểm chứng

Khi gọi `fetchProfile()`: console có log `GET ... -> 200` và JSON body.

Không crash.

Lưu ý cho SV: JSONPlaceholder là API fake, PUT chỉ "giả lập thành công", không lưu thật.

6) BƯỚC 6 — Repository (tách tầng cho sạch)

Tạo file:

lib/repositories/profile_repository.dart

```
import '../models/profile_model.dart';
import '../services/api_service.dart';

class ProfileRepository {
  final ApiService _api;

  ProfileRepository({ApiService? api}) : _api = api ?? ApiService();

  Future<Profile> getProfile() => _api.fetchProfile();

  Future<void> updateProfile(Profile profile) => _api.updateProfile(profile);
}
```

Kiểm chứng

Không lỗi compile.

7) BƯỚC 7 — Draft Service (SharedPreferences)

Tạo file:

lib/services/draft_service.dart

```
import 'dart:convert';
import 'package:shared_preferences/shared_preferences.dart';

import '../models/profile_model.dart';

class DraftService {
  static const String draftKey = "profile_draft";

  Future<void> saveDraft(Profile profile) async {
    final prefs = await SharedPreferences.getInstance();
    final raw = jsonEncode(profile.toJson());
    await prefs.setString(draftKey, raw);

    // ignore: avoid_print
    print("Draft saved: $raw");
  }

  Future<Profile?> loadDraft() async {
```


PhucPT10@fpt.edu.vn

```
    final prefs = await SharedPreferences.getInstance();
    final raw = prefs.getString(draftKey);

    // ignore: avoid_print
    print("Draft loaded raw: $raw");

    if (raw == null) return null;

    final jsonMap = jsonDecode(raw) as Map<String, dynamic>;
    return Profile.fromJson(jsonMap);
  }

  Future<void> clearDraft() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.remove(draftKey);

    // ignore: avoid_print
    print("Draft cleared");
  }
}
```

Kiểm chứng

Sau khi save, console in "Draft saved..."

Load được draft (sẽ test ở bước UI).

8) BƯỚC 8 — UI Profile Screen (Form + Controller + Focus + Validation + Scroll)

Tạo màn hình chính. Đây là phần quan trọng, mình viết **đầy đủ** để SV chỉ cần dán và chạy.

lib/screens/profile_screen.dart

```
import 'dart:async';

import 'package:flutter/material.dart';

import '../models/profile_model.dart';
import '../repositories/profile_repository.dart';
import '../services/draft_service.dart';

class ProfileScreen extends StatefulWidget {
  const ProfileScreen({super.key});

  @override
  State<ProfileScreen> createState() => _ProfileScreenState();
}

class _ProfileScreenState extends State<ProfileScreen> {
  final _formKey = GlobalKey<FormState>();

  final _nameCtrl = TextEditingController();
  final _emailCtrl = TextEditingController();
  final _phoneCtrl = TextEditingController();

  final _nameFocus = FocusNode();
  final _emailFocus = FocusNode();
  final _phoneFocus = FocusNode();
```

```
final _repository = ProfileRepository();
final _draftService = DraftService();

bool _loading = false;
String? _errorText;

// Chống save draft quá dày: debounce 400ms
Timer? _draftDebounce;

@override
void initState() {
  super.initState();
  _bootstrap();
}

Future<void> _bootstrap() async {
  setState(() {
    _loading = true;
    _errorText = null;
  });

  try {
    // 1) Ưu tiên draft trước (nếu có)
    final draft = await _draftService.loadDraft();
    if (draft != null) {
      _applyProfileToForm(draft);
    } else {
      // 2) Không c□□ draft => gọi API lấy profile
      final profile = await _repository.getProfile();
      _applyProfileToForm(profile);
    }
  } catch (e) {
    setState(() {
      _errorText = "Không tải được dữ liệu. Thử lại.\nChi tiết: $e";
    });
  }

  setState(() => _loading = false);
}

void _applyProfileToForm(Profile p) {
  _nameCtrl.text = p.name;
  _emailCtrl.text = p.email;
  _phoneCtrl.text = p.phone;
}

Profile _currentProfile() {
  return Profile(
    name: _nameCtrl.text.trim(),
    email: _emailCtrl.text.trim(),
    phone: _phoneCtrl.text.trim(),
  );
```


PhucPT10@fpt.edu.vn

```
}

// Validation đơn giản, phù hợp SV yếu
String? _validateRequired(String? value) {
  if (value == null || value.trim().isEmpty) return "Không được để trống";
  return null;
}

String? _validateEmail(String? value) {
  final v = (value ?? '').trim();
  if (v.isEmpty) return "Không được để trống";
  // Regex email đơn giản
  final ok = RegExp(r'^[^@]+@[^@]+\.[^@]+$').hasMatch(v);
  if (!ok) return "Email không hợp lệ (ví dụ: a@b.com)";
  return null;
}

String? _validatePhone(String? value) {
  final v = (value ?? '').trim();
  if (v.isEmpty) return "Không được để trống";
  // Cho số, dấu cách, +, -, ()
  final ok = RegExp(r'^[0-9+\-\s()]{8,}$').hasMatch(v);
  if (!ok) return "SĐT không hợp lệ (ít nhất 8 ký tự số)";
  return null;
}

void _scheduleDraftSave() {
  _draftDebounce?.cancel();
  _draftDebounce = Timer(const Duration(milliseconds: 400), () async {
    try {
      await _draftService.saveDraft(_currentProfile());
    } catch (e) {
      // ignore: avoid_print
      print("Save draft error: $e");
    }
  });
}

Future<void> _submit() async {
  // 1) Đóng keyboard
  FocusScope.of(context).unfocus();

  // 2) Validate
  final ok = _formKey.currentState?.validate() ?? false;
  if (!ok) return;

  setState(() {
    _loading = true;
    _errorText = null;
  });

  try {
    await _repository.updateProfile(_currentProfile());
```


PhucPT10@fpt.edu.vn

```
      await _draftService.clearDraft();

      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text("Cập nhật thành công")),
      );
    } catch (e) {
      setState(() {
        _errorText = "Cập nhật thất bại.\nChi tiết: $e";
      });

      if (!mounted) return;
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text("Cập nhật thất bại")),
      );
    }

    if (mounted) {
      setState(() => _loading = false);
    }
  }

  @override
  void dispose() {
    _draftDebounce?.cancel();

    _nameCtrl.dispose();
    _emailCtrl.dispose();
    _phoneCtrl.dispose();

    _nameFocus.dispose();
    _emailFocus.dispose();
    _phoneFocus.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final body = _loading
        ? const Center(child: CircularProgressIndicator())
        : _errorText != null
            ? _ErrorView(
                message: _errorText!,
                onRetry: _bootstrap,
              )
            : _buildForm();

    return Scaffold(
      appBar: AppBar(
        title: const Text("Smart Profile Editor"),
        actions: [
          IconButton(
            onPressed: _loading ? null : _bootstrap,
```


PhucPT10@fpt.edu.vn

```
            icon: const Icon(Icons.refresh),
          ),
        ],
      ),
      body: SafeArea(child: body),
    );
  }

  Widget _buildForm() {
    return GestureDetector(
      onTap: () => FocusScope.of(context).unfocus(),
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Form(
          key: _formKey,
          child: Column(
            children: [
              TextFormField(
                controller: _nameCtrl,
                focusNode: _nameFocus,
                decoration: const InputDecoration(
                  labelText: "Họ tên",
                  border: OutlineInputBorder(),
                ),
                textInputAction: TextInputAction.next,
                validator: _validateRequired,
                onFieldSubmitted: (_) =>
FocusScope.of(context).requestFocus(_emailFocus),
                onChanged: (_) => _scheduleDraftSave(),
              ),
              const SizedBox(height: 12),

              TextFormField(
                controller: _emailCtrl,
                focusNode: _emailFocus,
                decoration: const InputDecoration(
                  labelText: "Email",
                  border: OutlineInputBorder(),
                ),
                keyboardType: TextInputType.emailAddress,
                textInputAction: TextInputAction.next,
                validator: _validateEmail,
                onFieldSubmitted: (_) =>
FocusScope.of(context).requestFocus(_phoneFocus),
                onChanged: (_) => _scheduleDraftSave(),
              ),
              const SizedBox(height: 12),

              TextFormField(
                controller: _phoneCtrl,
                focusNode: _phoneFocus,
                decoration: const InputDecoration(
                  labelText: "Số điện thoại",
```

```
                  border: OutlineInputBorder(),
                ),
                keyboardType: TextInputType.phone,
                textInputAction: TextInputAction.done,
                validator: _validatePhone,
                onFieldSubmitted: (_) => _submit(),
                onChanged: (_) => _scheduleDraftSave(),
              ),
              const SizedBox(height: 16),

              SizedBox(
                width: double.infinity,
                child: ElevatedButton(
                  onPressed: _loading ? null : _submit,
                  child: const Text("Lưu cập nhật"),
                ),
              ),

              const SizedBox(height: 8),
              Text(
                "Gợi ý: nhập thử sai email/phone để xem validation.\n"
                "Thoát app mở lại để kiểm tra Draft.",
                style: Theme.of(context).textTheme.bodySmall,
                textAlign: TextAlign.center,
              )
            ],
          ),
        ),
      ),
    );
  }
}

class _ErrorView extends StatelessWidget {
  final String message;
  final Future<void> Function() onRetry;

  const _ErrorView({required this.message, required this.onRetry});

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(16),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          Text(
            message,
            style: const TextStyle(color: Colors.red),
            textAlign: TextAlign.center,
          ),
          const SizedBox(height: 12),
          ElevatedButton(
```


PhucPT10@fpt.edu.vn

```
            onPressed: () => onRetry(),
            child: const Text("Thử lại"),
          ),
        ],
      ),
    );
  }
}
```

Kiểm chứng từng ý (rất quan trọng)

Load API

Lần đầu chạy, form có dữ liệu (từ user/1 của JSONPlaceholder: name/email/phone).

Console thấy log GET.

Validation

Xóa email → bấm "Lưu cập nhật" → báo "Không được để trống".

Nhập abc → báo "Email không hợp lệ".

Focus & Keyboard

Nhấn "Next" trên bàn phím: nhảy đúng field tiếp theo.

Field cuối nhấn "Done": gọi submit.

Draft

Gõ sửa tên/email/phone.

Thoát app hoàn toàn (close).

Mở lại app: dữ liệu **vẫn là dữ liệu bạn gõ**, không quay về API.

Console thấy "Draft saved" và "Draft loaded".

Submit + Clear draft

Bấm "Lưu cập nhật" → hiện snackbar "Cập nhật thành công".

Thoát app mở lại: vì draft đã clear nên app sẽ load API lại (quay về dữ liệu API).

9) BƯỚC 9 — Nối màn hình vào main.dart

Sửa `lib/main.dart` như sau:

lib/main.dart

```
import 'package:flutter/material.dart';
import 'screens/profile_screen.dart';

void main() {
  runApp(const SmartProfileEditorApp());
}

class SmartProfileEditorApp extends StatelessWidget {
  const SmartProfileEditorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Smart Profile Editor',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorSchemeSeed: Colors.blue,
      ),
      home: const ProfileScreen(),
```


PhucPT10@fpt.edu.vn

```
    );
  }
}
```

Kiểm chứng

App mở thẳng vào màn hình "Smart Profile Editor".

10) Checklist tổng hợp để chấm bài (SV tự đối chiếu)

SV quay video/ghi màn hình chứng minh các ý sau:

Mở app lần đầu → thấy dữ liệu load từ API (GET)
Nhập sai email/phone → không cho submit
Next/Done trên keyboard hoạt động đúng
Nhập dữ liệu → thoát app → mở lại vẫn còn (Draft)
Bấm Lưu → loading → success → draft bị clear (mở lại không còn draft)

11) Lỗi phổ biến & cách sửa (bản "chắc chắn gặp")

Keyboard che nút
Sửa: đã bọc `SingleChildScrollView` + `SafeArea` (đã làm trong code)
API gọi nhiều lần
Nguyên nhân: gọi trong `build()`
Sửa: gọi trong `initState()` (đã làm `_bootstrap()`)
Draft save quá nhiều lần
Nguyên nhân: onChanged gọi liên tục
Sửa: debounce Timer (đã làm `_scheduleDraftSave()`)
Memory leak
Nguyên nhân: quên `dispose()`
Sửa: dispose controller/focus (đã làm)

PhucPT10@fpt.edu.vn

12) CÁC GỢI Ý BỔ SUNG, NÂNG CAO

2 NÂNG CẤP API LAYER (PRODUCTION-READY)

Mục tiêu

Thêm timeout
Xử lý status code đúng chuẩn HTTP
Tách error type rõ ràng
Không throw Exception chung chung

12.1 Thêm Timeout cho HTTP

Trong `api_service.dart`, sửa:

```
final response = await http.get(uri)
    .timeout(const Duration(seconds: 10));
```

và

```
final response = await http.put(
  uri,
  body: jsonEncode(profile.toJson()),
  headers: {'Content-Type': 'application/json'},
).timeout(const Duration(seconds: 10));
```

Lý do

Nếu mạng treo:

App không bị chờ vô hạn
Chủ động show error

12.2 Chuẩn hóa kiểm tra Status Code

Thay:

```
if (response.statusCode != 200)
```

Bằng:

```
if (response.statusCode < 200 || response.statusCode >= 300)
```

Vì sao?

HTTP success range là 200–299:

200 OK
201 Created
204 No Content

12.3 Tạo Custom Exception

Tạo file:

```
lib/core/exceptions.dart
class ApiException implements Exception {
  final String message;
  final int? statusCode;

  ApiException(this.message, {this.statusCode});

  @override
  String toString() {
    return "ApiException($statusCode): $message";
  }
}
```

Trong `ApiService`, thay:

```
throw Exception("Failed to load profile");
```

Bằng:

PhucPT10@fpt.edu.vn

```
throw ApiException(
  "Failed to load profile",
  statusCode: response.statusCode,
);
```

13 DIRTY STATE DETECTION (PHÁT HIỆN FORM ĐÃ SỬA)

Vấn đề hiện tại

User sửa dữ liệu → thoát app → draft lưu OK.
Nhưng nếu user:

Vào app
Không sửa gì
Bấm submit

→ vẫn gọi PUT (không cần thiết).

13.1 Thêm biến lưu originalProfile

Trong `_ProfileScreenState`:

```
Profile? _originalProfile;
```

Sau khi load thành công:

```
_originalProfile = p;
```

13.2 Tạo hàm so sánh

```
bool get _isDirty {
  if (_originalProfile == null) return false;
  return _originalProfile != _currentProfile();
}
```

Bạn cần override == trong model:

```
@override
bool operator ==(Object other) {
  if (identical(this, other)) return true;
  return other is Profile &&
      other.name == name &&
      other.email == email &&
      other.phone == phone;
}

@override
int get hashCode => name.hashCode ^ email.hashCode ^ phone.hashCode;
```

13.3 Disable nút submit nếu không dirty

```
onPressed: _loading || !_isDirty ? null : _submit,
```

14 AUTO VALIDATE MODE

Hiện tại chỉ validate khi submit.

Nâng cấp UX

Trong Form:

```
Form(
  key: _formKey,
  autovalidateMode: AutovalidateMode.onUserInteraction,
```

Hiệu quả

Gõ sai → báo ngay
Không cần chờ submit

PhucPT10@fpt.edu.vn

1 5 CONFIRM BEFORE EXIT (BẢO VỆ DỮ LIỆU)

Vấn đề

User sửa form nhưng chưa submit → bấm back → mất dữ liệu.

15.1 Dùng WillPopScope

Bọc Scaffold bằng:

```
return WillPopScope(
  onWillPop: _onWillPop,
  child: Scaffold(
```

15.2 Thêm logic

```
Future<bool> _onWillPop() async {
  if (!_isDirty) return true;

  final result = await showDialog<bool>(
    context: context,
    builder: (_) => AlertDialog(
      title: const Text("Bạn chưa lưu"),
      content: const Text("Bạn có chắc muốn thoát không?"),
      actions: [
        TextButton(
          onPressed: () => Navigator.pop(context, false),
          child: const Text("Ở lại"),
        ),
        TextButton(
          onPressed: () => Navigator.pop(context, true),
          child: const Text("Thoát"),
        ),
      ],
    ),
  );

  return result ?? false;
}
```

---

1 6 OFFLINE DETECTION (TÙY CHỌN)

Thêm package:

```
connectivity_plus:
```

Kiểm tra trước khi submit:

```
final connectivity = await Connectivity().checkConnectivity();
if (connectivity == ConnectivityResult.none) {
  setState(() {
    _errorText = "Không có kết nối mạng";
  });
  return;
}
```

1 7 AVATAR UPLOAD (MỞ RỘNG UI)

Mục tiêu

Thêm ảnh đại diện
Cho chọn từ gallery
Hiển thị preview

Thêm package:

PhucPT10@fpt.edu.vn

```
image_picker:
```
Trong UI:
```
CircleAvatar(
  radius: 40,
  backgroundImage: _avatarFile != null
      ? FileImage(_avatarFile!)
      : null,
  child: _avatarFile == null
      ? const Icon(Icons.person, size: 40)
      : null,
),
```

18 REFACTOR SANG CHANGE NOTIFIER (LEVEL CAO HƠN)

Tạo:
```
profile_viewmodel.dart
```
Di chuyển:

- loading
- error
- submit
- bootstrap
- draft

ra khỏi UI.

UI chỉ còn:
```
Consumer<ProfileViewModel>(
  builder: ...
)
```

19 OPTIMISTIC UPDATE

Hiện tại:

- Chờ API xong mới show success

Nâng cấp:

- Update UI ngay
- Nếu fail → rollback lại originalProfile

20 THÊM UNIT TEST (RẤT QUAN TRỌNG)

20.1 Test Model

- fromJson
- toJson
- copyWith

20.2 Test Repository

Mock ApiService.

20.3 Test DraftService

Dùng SharedPreferences.setMockInitialValues({});

21 THÊM LOADING OVERLAY THAY VÌ REBUILD TOÀN BODY

Thay:
```
_loading ? CircularProgressIndicator()
```
Bằng:
```
Stack(
  children: [
    _buildForm(),
    if (_loading)
      const Center(child: CircularProgressIndicator()),
```


PhucPT10@fpt.edu.vn

```
    ],
)
```

UX tốt hơn.

KẾT LUẬN MỨC ĐỘ MỞ RỘNG

Level	Nội dung
Intermediate	Dirty state, Autovalidate
Advanced	Custom exception, Timeout
Senior-ish	Optimistic update
Production-ready	Offline detect, Unit test

PhucPT10@fpt.edu.vn

FOUNDATION GLOSSARY – SLOT 12

I. FORM & VALIDATION

Thuật ngữ	Định nghĩa	Giải thích dễ hiểu
Form	Widget quản lý nhiều FormField	Giống như "container" gom tất cả input lại
FormState	Trạng thái nội bộ của Form	Cho phép gọi validate(), save()
GlobalKey	Key truy cập state của widget	Dùng để truy cập FormState
TextField	Widget nhập liệu cơ bản	Không có validation tích hợp
TextFormField	TextField + FormField	Có validator sẵn
Validator	Hàm kiểm tra dữ liệu	Trả về lỗi nếu dữ liệu sai
autovalidateMode	Tự động validate	Validate khi user nhập
Cross-field validation	Kiểm tra nhiều field liên quan	Ví dụ password & confirm
Draft Save	Lưu tạm dữ liệu form	Tránh mất dữ liệu khi thoát
Debounce	Trì hoãn xử lý một hành động	Tránh save mỗi ký tự

II. FOCUS & KEYBOARD

Thuật ngữ	Định nghĩa	Giải thích dễ hiểu
FocusNode	Quản lý trạng thái focus	Biết field nào đang được chọn
FocusScope	Nhóm các FocusNode	Điều hướng giữa field
requestFocus()	Yêu cầu focus	Chuyển con trỏ sang field khác
nextFocus()	Chuyển sang field kế	Dùng khi nhấn Next
TextInputAction	Hành động của nút keyboard	next, done, send
Keyboard Inset	Khoảng trống do bàn phím chiếm	Dùng MediaQuery để tính toán
resizeToAvoidBottomInset	Điều chỉnh layout khi keyboard mở	Tránh bị che UI
Lifecycle (initState/dispose)	Vòng đời widget	Phải dispose FocusNode

III. HTTP & NETWORKING

Thuật ngữ	Định nghĩa	Giải thích dễ hiểu
HTTP	Giao thức truyền dữ liệu web	App giao tiếp server
Request	Yêu cầu gửi đi	App gửi lên server
Response	Phản hồi nhận về	Server trả kết quả
GET	Lấy dữ liệu	Không thay đổi dữ liệu
POST	Tạo mới dữ liệu	Gửi form lên server
PUT	Cập nhật dữ liệu	Chỉnh sửa
DELETE	Xóa dữ liệu	Xóa record
Status Code	Mã trạng thái	200 OK, 401 Unauthorized
Timeout	Hết thời gian chờ	Mạng chậm
Retry	Thử lại	Khi request fail
Stateless	Không lưu trạng thái	Mỗi request độc lập
Header	Thông tin bổ sung	Content-Type, Authorization

IV. ASYNCHRONOUS PROGRAMMING

Thuật ngữ	Định nghĩa	Giải thích dễ hiểu
Future	Giá trị sẽ có trong tương lai	Kết quả async
async	Đánh dấu hàm bất đồng bộ	Cho phép await
await	Chờ Future hoàn thành	Không block UI
Event Loop	Cơ chế xử lý async	Dart xử lý task nền

PhucPT10@fpt.edu.vn

try/catch	Bắt lỗi	Tránh crash
Exception	Lỗi runtime	Lỗi khi chạy

V. JSON & DATA MODELING

Thuật ngữ	Định nghĩa	Giải thích dễ hiểu
JSON	Định dạng dữ liệu text	Dùng trao đổi dữ liệu
Serialization	Object → JSON	Chuẩn bị gửi server
Deserialization	JSON → Object	Parse dữ liệu nhận
jsonDecode()	Hàm parse JSON	Trả về Map
Map<String, dynamic>	Kiểu dữ liệu key-value	Cấu trúc JSON trong Dart
Model Class	Lớp đại diện dữ liệu	Giúp type-safe
fromJson()	Tạo object từ JSON	Factory constructor
toJson()	Chuyển object thành JSON	Chuẩn bị gửi đi
Nested JSON	JSON lồng nhau	Object trong object
Null-safety	An toàn với null	Tránh crash

VI. KIẾN TRÚC & PHÂN TẦNG

Thuật ngữ	Định nghĩa	Giải thích dễ hiểu
Repository Pattern	Tầng trung gian data	UI không gọi API trực tiếp
Service Layer	Xử lý network	Tách khỏi UI
Separation of Concerns	Tách trách nhiệm	Mỗi phần làm đúng việc
Layered Architecture	Kiến trúc phân tầng	UI → Repository → Service
Single Responsibility Principle	Một lớp một nhiệm vụ	Dễ bảo trì
Offline-first	Ưu tiên local trước server	Giảm phụ thuộc mạng

CÁC THUẬT NGỮ NÂNG CAO (DÀNH CHO SINH VIÊN KHÁ – GIỎI)

Thuật ngữ	Ý nghĩa
Idempotent API	Gọi nhiều lần vẫn cùng kết quả
Interceptor	Chặn request để thêm header
Optimistic Update	Cập nhật UI trước khi server phản hồi
Graceful Degradation	Hoạt động tốt ngay cả khi lỗi
Memory Leak	Rò rỉ bộ nhớ
State Synchronization	Đồng bộ trạng thái local & server

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU CHUYÊN SÂU

1 Phân tích kiến trúc lưu Draft trong ứng dụng lớn

Prompt:
So sánh các cách triển khai Draft Save trong Flutter: lưu bằng `SharedPreferences`, `Hive`, `SQLite`, và lưu tạm trên server.
Phân tích trade-off về hiệu năng, bảo mật, khả năng mở rộng, đồng bộ đa thiết bị và tính nhất quán dữ liệu.
Trong dự án thương mại (ví dụ app TMĐT), bạn sẽ chọn giải pháp nào và vì sao?
🎯 Mục tiêu: Tư duy kiến trúc & lựa chọn công nghệ.

2 Phân tích vòng đời Form và quản lý State chuyên nghiệp

Prompt:
Phân tích lifecycle của Form trong Flutter khi kết hợp với `Provider`, `Riverpod` hoặc `Bloc`.
Làm rõ sự khác biệt giữa state tạm thời (UI state) và state nghiệp vụ (business state).
Khi nào nên tách Form logic ra khỏi Widget?
🎯 Mục tiêu: Hiểu clean architecture & separation of concerns.

3 Thiết kế UX chuyên sâu cho Focus & Keyboard

Prompt:
Thiết kế luồng Focus & Keyboard Actions cho một form đăng ký 10 trường dữ liệu theo chuẩn UX chuyên nghiệp.
Phân tích cách xử lý:
- Validation realtime
- Auto-scroll khi bàn phím mở
- Submit bằng Enter
- Accessibility (screen reader)

🎯 Mục tiêu: UX thinking + kỹ thuật điều khiển FocusNode chuyên sâu.

4 Phân tích hiệu năng khi parse JSON lớn

Prompt:
Khi nhận một JSON lớn (1000+ items), phân tích các vấn đề hiệu năng khi parse trên main isolate.
So sánh:
- Parse đồng bộ
- Sử dụng `compute()`
- Sử dụng isolate riêng
  - Đánh giá tác động đến UI responsiveness.

🎯 Mục tiêu: Hiểu concurrency và isolate trong Flutter.

5 Thiết kế HTTP Layer theo chuẩn Clean Architecture

Prompt:
Thiết kế một HTTP layer chuẩn production gồm:
- API service
- Repository
- Error handling
- Response mapping
  - Phân tích cách tách domain model và DTO model.

🎯 Mục tiêu: Nâng cấp tư duy từ demo lên production-level architecture.

PhucPT10@fpt.edu.vn

6 Phân tích chiến lược xử lý lỗi mạng chuyên nghiệp

Prompt:
Xây dựng chiến lược xử lý lỗi cho HTTP request gồm:

- Timeout
- Retry
- Token expired
- No internet
- Server 500

Thiết kế UX hiển thị lỗi phù hợp từng trường hợp.

🎯 Mục tiêu: Reliability & fault tolerance.

7 So sánh Manual JSON Parsing vs Code Generation

Prompt:
So sánh manual parsing với `json_serializable`, `freezed`, `built_value`.
Phân tích:

- Maintainability
- Type safety
- Error prevention
- Hiệu suất build

Trong dự án lớn 100+ model, bạn chọn cách nào?

🎯 Mục tiêu: Đánh giá tooling và automation.

8 Thiết kế Offline-first Strategy cho Form & HTTP

Prompt:
Thiết kế chiến lược Offline-first cho một form tạo đơn hàng:

- Lưu local trước
- Đồng bộ khi có mạng
- Conflict resolution
- Trạng thái đồng bộ (sync status UI)

Phân tích rủi ro và cách giảm thiểu.

🎯 Mục tiêu: Tư duy hệ thống & consistency.

9 Phân tích bảo mật khi gửi dữ liệu Form qua HTTP

Prompt:
Phân tích các rủi ro bảo mật khi gửi form data:

- Man-in-the-middle
- Token storage
- Sensitive data
- Replay attack

Đề xuất giải pháp bảo vệ trên mobile app.

🎯 Mục tiêu: Security mindset trong mobile development.

10 Thiết kế Integration Testing cho Form + HTTP

Prompt:
Xây dựng chiến lược test cho:

- Form validation
- Draft save
- HTTP request
- JSON parsing

Phân tích cách mock API và viết integration test với `flutter_test` hoặc `mocktail`.

🎯 Mục tiêu: Nâng cấp từ code chạy được → code có thể kiểm chứng được.

PhucPT10@fpt.edu.vn

10 PROMPT TỰ NGHIÊN CỨU THỰC TIỄN CHUYÊN NGHIỆP

NHÓM 1: FORM & DRAFT SAVE TRONG DỰ ÁN LỚN

1 Thiết kế hệ thống Draft Save cho ứng dụng có hàng triệu người dùng

Prompt:
Thiết kế cơ chế Draft Save cho một ứng dụng thương mại điện tử cho phép người dùng tạo đơn hàng nhiều bước.
Phân tích:
- Lưu local hay server?
- Đồng bộ đa thiết bị
- Xử lý conflict khi chỉnh sửa trên 2 thiết bị
- Khi nào nên xóa draft?
  - Đề xuất kiến trúc tổng thể và flow dữ liệu.

2 Phân tích chiến lược Auto-save theo thời gian vs theo sự kiện

Prompt:
So sánh hai chiến lược auto-save:
- Lưu mỗi khi người dùng nhập
- Lưu theo chu kỳ (debounce 2–5 giây)
  - Phân tích tác động đến:
- Hiệu năng
- Pin
- Trải nghiệm người dùng
- Server load
  - Đề xuất chiến lược tối ưu cho production app.

3 Thiết kế Form Validation cho hệ thống nhiều bước (multi-step form)

Prompt:
Xây dựng chiến lược validation cho form 5 bước gồm:
- Validate từng bước
- Validate tổng thể trước submit
- Hiển thị lỗi UX-friendly
  - Phân tích cách quản lý state giữa các bước.

4 Xử lý Form phức tạp có dynamic fields

Prompt:
Thiết kế cách xử lý form có trường dữ liệu động (ví dụ thêm nhiều địa chỉ giao hàng).
Phân tích:
- Quản lý controller động
- Xử lý dispose
- Serialize dữ liệu
- Lưu draft cho cấu trúc động

5 Phân tích memory leak trong Form lớn

Prompt:
Khi form có 30+ TextEditingController và FocusNode, phân tích nguy cơ memory leak.
Làm rõ:
- Khi nào cần dispose
- Cách refactor để tránh leak
- Cách kiểm tra bằng DevTools

PhucPT10@fpt.edu.vn

⌨ NHÓM 2: FOCUS & KEYBOARD TRONG ỨNG DỤNG CHUYÊN NGHIỆP

6 Thiết kế trải nghiệm nhập liệu chuyên nghiệp cho mobile banking

Prompt:
Thiết kế hệ thống Focus & Keyboard cho form chuyển tiền ngân hàng.
Phân tích:
- Keyboard type phù hợp từng trường
- Chuyển focus tự động
- Submit bằng Enter
- Accessibility & hỗ trợ người khuyết tật

7 Xử lý vấn đề bàn phím che nội dung trên nhiều thiết bị

Prompt:
Phân tích các giải pháp xử lý bàn phím che nội dung:
- resizeToAvoidBottomInset
- SingleChildScrollView
- ScrollController
- MediaQuery

So sánh ưu nhược điểm trong app production.

8 Phân tích UX khi dùng External Keyboard (tablet, desktop)

Prompt:
Khi app chạy trên tablet hoặc desktop có bàn phím vật lý, thiết kế keyboard actions chuyên nghiệp:
- Tab navigation
- Enter submit
- Escape cancel

Đề xuất cách triển khai trong Flutter.

🌐 NHÓM 3: HTTP REQUEST & NETWORK LAYER PRODUCTION

9 Thiết kế HTTP Client Layer chuẩn doanh nghiệp

Prompt:
Thiết kế HTTP client layer gồm:
- Interceptor
- Logging
- Retry
- Token refresh
- Centralized error handling

Vẽ sơ đồ kiến trúc và giải thích luồng request-response.

10 Phân tích chiến lược retry & exponential backoff

Prompt:
Phân tích cách triển khai retry với exponential backoff.
Khi nào nên retry?
Khi nào không nên retry (ví dụ 400)?
Tác động đến server và người dùng.

1 1 Thiết kế cơ chế token refresh tự động

Prompt:
Phân tích cách triển khai auto refresh token khi 401 xảy ra.
Xử lý trường hợp:
- Nhiều request cùng hết hạn

PhucPT10@fpt.edu.vn

Token refresh thất bại
User logout an toàn

1 2 So sánh REST API và GraphQL trong bối cảnh mobile app

Prompt:
So sánh REST và GraphQL khi xây dựng mobile app có form submit và danh sách lớn.
Phân tích:
Overfetching
Underfetching
Caching
Complexity
Khi nào nên chọn giải pháp nào?

NHÓM 4: JSON PARSING & DATA MODELING Ở QUY MÔ LỚN

1 3 Thiết kế Data Model & DTO chuyên nghiệp

Prompt:
Phân tích cách tách:
API Response Model (DTO)
Domain Model
UI Model
Làm rõ vì sao không nên dùng trực tiếp JSON model cho UI.

1 4 Xử lý JSON phức tạp & nested data

Prompt:
Phân tích cách xử lý JSON có:
Nested objects
List lồng nhau
Optional field
Null safety
So sánh manual parsing và code generation.

1 5 Phân tích hiệu năng và tối ưu parsing dữ liệu lớn

Prompt:
Khi server trả về 5000 record JSON, phân tích:
UI jank
Blocking main isolate
Memory usage
Đề xuất giải pháp:
Isolate
Pagination
Lazy loading
Streaming response

PhucPT10@fpt.edu.vn